

changepointGA: An R Package for Fast Changepoint Detection via Genetic Algorithms

by Mo Li and QiQi Lu

Abstract Detecting changepoints requires searching a model space that grows exponentially with the length of the time series, making exhaustive enumeration computationally infeasible. Genetic algorithms provide a stochastic way to identify the structural changes: a population of candidate models evolves via selection, crossover, and mutation operators until it converges on one changepoint model that balances the goodness-of-fit with parsimony. The R package `changepointGA` encodes each candidate model as an integer chromosome vector and supports both the basic single-population model and the island model. Parallel computing is implemented on multi-core hardware to further accelerate computation. Users may supply custom fitness functions or genetic operators, while a streamlined interface supports routine analyses. Extensive simulations demonstrate that our package runs significantly faster than binary-encoded genetic algorithm alternatives. Additionally, this package can simultaneously locate changepoints, estimate their effects, and select time series model orders, such as autoregressive and moving-average orders. Applications to array-based comparative genomic hybridization data and a century-long temperature series further highlight the package's value in biological and climate research.

1 Introduction

In statistical analysis and time series modeling, a changepoint refers to a time at which there is a change in the underlying statistical properties or behavior of the data. It could indicate a change in the mean (Robbins et al., 2011; Aue and Horváth, 2013), variance (Gombay et al., 1996; Aue and Horváth, 2013; Chapman et al., 2020), or autocorrelation structure, such as a shift in the lag dependence of the series (Davis et al., 2006; Aue and Horváth, 2013; Dette et al., 2019). Detecting changepoints is crucial across various fields (Chen and Gupta, 2000) such as finance (Andreou and Ghysels, 2002), economics (Bai and Perron, 1998; Zeileis et al., 2005), environmental science (Li and Lu, 2022), and signal processing (Gavrin et al., 2018), as it helps identify shifts in trends, detect anomalies, and make predictions.

Since Page (1955) first explored changepoint detection on independently and identically distributed (IID) data, the literature on this topic has grown significantly. A common strategy is to assume a parametric regression model with potential changepoints in the observed sequence and then compare the fit across different configurations, varying both the number and the locations of changepoints. The best-fitting model is then used to estimate the changepoints (Aue and Horváth, 2013; Shi et al., 2022). However, obtaining this model requires fitting all candidate models across every possible changepoint configuration, which is computationally demanding, particularly for long time series. For instance, given a time series with N observations and assuming at most one changepoint (AMOC), there are $N - 1$ candidate configurations, each requiring a separate model fit. When multiple changepoints are allowed, the number of candidate configurations increases exponentially to 2^{N-1} (Lee et al., 2020). This exponential growth of the search space renders the problem computationally intractable, and consequently, exhaustive search becomes impractical even for moderate values of N .

Rather than exhaustively searching all possible changepoint models to identify the globally optimal model, genetic algorithms aim to efficiently explore the configuration space and find a solution that provides the optimal fit. Originally introduced by Holland (1975) as a stochastic search technique, genetic algorithms have proven effective for solving optimization problems involving a dynamically changing number of parameters. They operate by mimicking the evolutionary process in nature, progressively refining candidate solutions across successive generations. The GA optimization package in R (R Core Team, 2024), developed by Scrucca (2013), serves as a general-purpose framework for implementing genetic algorithms in search spaces comprising both continuous- and discrete-valued solutions. Improvements such as the island model genetic algorithm and parallel computation were later incorporated into the GA package (Scrucca, 2017). Within this framework, a common approach to representing a changepoint configuration is to encode it as a binary string of length N (Shi et al., 2022). In this representation, a value of one in the binary string indicates the presence of a changepoint at the corresponding time point, whereas a value of zero indicates its absence. However, this encoding method has three notable limitations in the context of changepoint detection.

First, although genetic algorithms with binary encoding reduce the need to fit every possible

changepoint model, they represent candidate solutions as vectors of length N , resulting in high-dimensional chromosomes, especially for long time series. Second, binary encoding permits unrealistic changepoint configurations, such as placing a changepoint at $t = 1$ or assigning two adjacent changepoints at times t and $t + 1$. Such configurations can lead to convergence issues during parameter estimation (e.g., computational singularities when inverting matrices), particularly in models with complex structures. Third, for time series models with changepoints, such as piecewise autoregressive moving-average (ARMA) processes (Davis et al., 2006; Shi et al., 2022), piecewise trend-stationary series with periodic autoregressive (PAR) innovations (Lu et al., 2010), the GA package requires users to pre-specify order parameters (i.e., AR, MA, or PAR orders) to run the algorithm (Shibata, 1976; Scrucca, 2017). These parameters define the number of past observations or residuals used to model temporal dependencies. When the true orders are unknown, a separate model selection step is required to estimate them. However, standard estimation methods often degrade unless the changepoint locations are known a priori. Ideally, a changepoint detection method should dynamically and simultaneously estimate both the model structure and the changepoint locations to enable more robust analysis.

Instead of using a purely binary chromosome representation, Davis et al. (2006) used a length- N encoding in which non-breakpoint locations are marked by -1 , whereas segment-start locations store the AR order of the corresponding segment. This representation enables the genetic algorithm to estimate changepoint locations and segment-specific AR orders simultaneously, but the chromosome length remains tied to the length of the time series. By contrast, Lu et al. (2010) adopted a more compact representation in which the number of changepoints and their locations are encoded directly as integer-valued vectors. This approach substantially reduces chromosome dimensionality and improves both computational and memory efficiency. Related genetic-algorithm encodings have also been applied to changepoint detection in subsequent work (Polushina and Sofronov, 2011; Lee et al., 2014; Li et al., 2026).

Building on this idea, `changepointGA` provides a specialized R framework for efficient changepoint detection in time series using genetic algorithms with a compact integer-encoded chromosome representation. In contrast to the general-purpose stochastic search framework provided by GA, `changepointGA` is designed specifically for changepoint analysis. Its main practical advantages include substantially reducing computational cost, enforcing a minimum segment-length constraint to exclude unrealistic nearby changepoints, and allowing time series model orders to be estimated jointly with changepoint locations.

The `changepointGA` package is publicly available from the Comprehensive R Archive Network (CRAN). In genetic algorithms, the fitness or objective function is typically problem-dependent. In the changepoint detection literature, commonly used objective functions include the Bayesian information criterion (BIC) (Yao, 1988), modified BIC (mBIC) (Zhang and Siegmund, 2007), and minimum description length (MDL) (Davis et al., 2006). Accordingly, `changepointGA` provides a flexible implementation of genetic algorithms that allows users to supply problem-specific fitness functions and to define customized genetic operators using either R or Rcpp. Although the algorithm is executed sequentially by default, the package also supports parallel computation to improve performance, particularly when fitness evaluations are computationally intensive.

Furthermore, `changepointGA` can be applied to a wide range of time series settings, including piecewise stationary ARMA processes and piecewise trend-stationary series. However, care is needed for integrated processes, such as random walks with drift or deterministic trends, because stochastic trends induced by unit roots may be mistaken for level shifts in finite samples. As a preliminary diagnostic step, users may consider conducting appropriate unit root tests before applying `changepointGA` (Perron, 1989; Zivot and Andrews, 1992).

The remainder of the paper is organized as follows. Section 2 provides a structured overview of genetic algorithms for changepoint detection. Section 3 introduces the main functions `cptga` and `cptga1` in `changepointGA`. Section 4 presents simulation studies for changepoint detection with and without ARMA order selection. Two real-world case studies are included in Section 5 to highlight the utility of the proposed framework. Finally, Section 6 summarizes the main contributions and offers concluding remarks.

2 Background

For any pre-specified time series model with a given set of changepoint locations, model fit is evaluated using a fitness function $Q(\theta)$, where $\theta = (s, \tau, \beta)^\top$ denotes the full parameter vector. Here, s represents the set of model hyperparameters, which may include the AR order, the MA order, or the periodic AR order in PAR models. The vector $\tau = \{\tau_1, \dots, \tau_m\}$ specifies the changepoint locations such that $1 \leq \tau_1 < \tau_2 < \dots < \tau_m < N$, with the number of changepoints m inferred as part of the estimation process. Once s and τ are determined, the remaining model parameters in β can be estimated through

standard model fitting procedures. Within this framework, changepoint detection is formulated as a combinatorial optimization problem: the goal is to minimize the scalar-valued objective function $Q(\theta)$ and thereby identify the optimal solution

$$\theta^* \equiv \arg \min_{\theta \in S} Q(\theta) = \{\theta^* \in S : Q(\theta^*) \leq Q(\theta), \forall \theta \in S\}, \quad (1)$$

where S denotes the feasible parameter space. Note that while the problem statement in Equation (1) revolves around minimizing $Q(\theta)$, it is trivial to transform the maximization objective into a minimization problem by simply adjusting the sign of the objective function. When estimating the optimal changepoint configuration along with the model orders and parameters, the dimensionality of θ varies and depends on the model order and the number of changepoints. This dynamic dimensionality adds further complexity to the challenge of changepoint detection.

The genetic algorithm operates as a stochastic search optimization method inspired by the principle of survival of the fittest individual in ecology. Each individual in the population is characterized by a chromosome that encodes a potential solution. These chromosomes contain values from θ within the feasible search space S for the optimization problem. Subsequently, the fitness of each individual (or chromosome) is assessed based on the objective function value $Q(\theta)$. In our `changepointGA` package, the chromosome representation is specified as a vector,

$$C = (m, s, \tau, N + 1)^T. \quad (2)$$

Note that if no model order selection is desired, then s is omitted and the genetic algorithm detects only the changepoints. This flexibility enables the package to accommodate diverse user requirements and problem settings, ensuring the genetic algorithm is precisely tailored to the optimization task. The changepoint locations in τ are encoded as integer values between 1 and $N - 1$, allowing the length of τ to vary dynamically with m . The value $N + 1$ is appended at the end of C to serve as a delimiter marking the boundary of the chromosome. The dimensionality of the model parameter vector β also varies, depending on the values specified in s and τ .

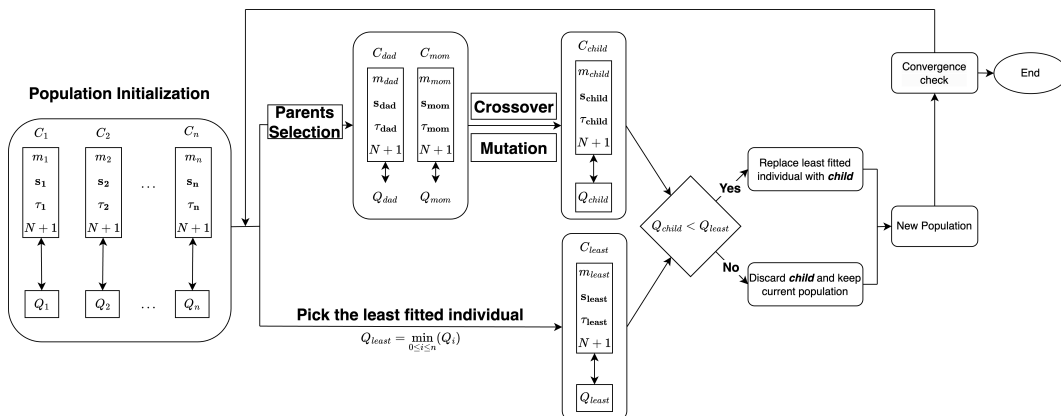


Figure 1: The flowchart of the basic genetic algorithm.

The genetic algorithm starts with the population initialization of size n . For each chromosome (individual), every gene (element) in s is randomly selected with equal probability from a user-specified candidate set. For example, for an ARMA setting, the AR and MA orders are randomly chosen from the candidate order set, $\{0, 1, 2, 3, 4\}$, which includes the commonly used values for AR and MA orders. The number of changepoints m and their locations τ are also randomly generated. For each chromosome, the specified model is then fit and its fitness evaluated with respect to the objective function $Q(\theta)$. Subsequently, the selection operator chooses pairs of individuals from the population according to their fitness values, using methods such as the linear ranking algorithm, with higher-fitness individuals (those with smaller $Q(\theta)$) having a greater probability of being selected as parents. These parents undergo the crossover operator, with a specified crossover probability, to produce offspring (children) for the next generation. Reflecting biological evolution, each offspring is also subject to a mutation operator that, with a relatively low probability, introduces random genes in its chromosome. This mutation helps the algorithm avoid becoming trapped in local optima. The offspring is then incorporated into the next population using methods such as the steady-state approach from Davis (1991). As shown in Figure 1, if the offspring exhibits superior fitness (i.e., a smaller $Q(\theta)$) than the least fit individual, it replaces that worst individual in the current generation. Otherwise, or if it duplicates an existing chromosome, it is discarded and a new offspring is produced.

via the selection–crossover–mutation cycle. As generations proceed, the population’s best fitness typically improves because fitter individuals are more likely to reproduce. This process repeats until specified termination or convergence criteria are met. The fittest individual in the final population is taken as the approximate optimizer of the objective, and its chromosome yields the estimate of θ , $\hat{\theta}$.

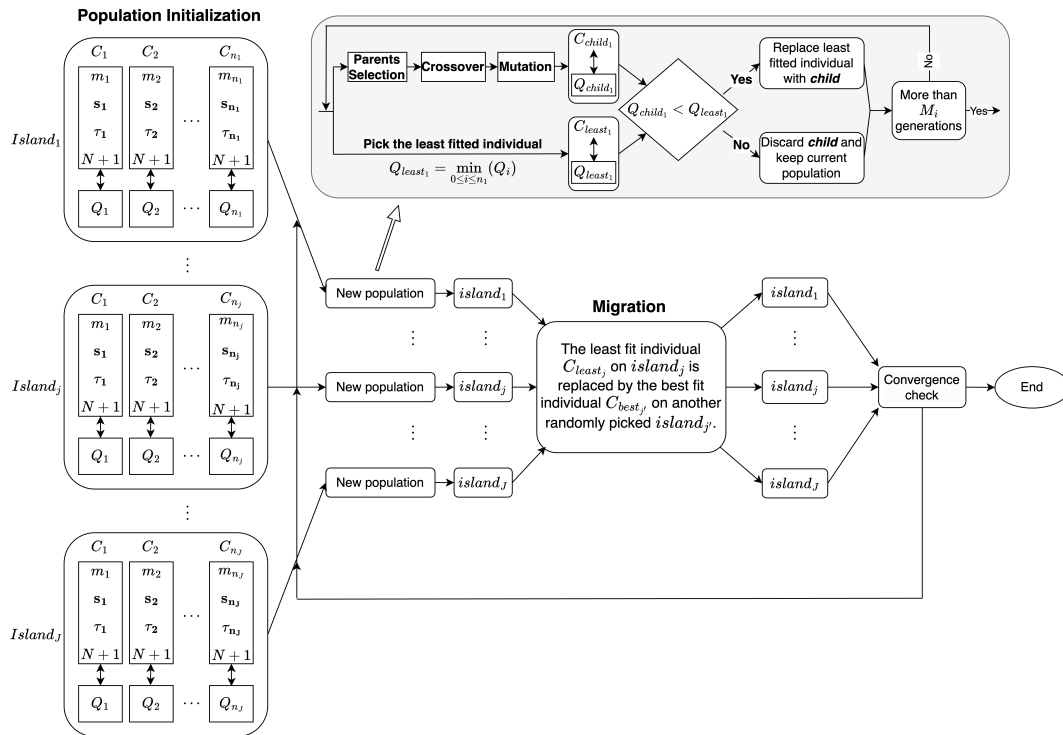


Figure 2: The flowchart of the island model genetic algorithm.

In addition to the standard single-population genetic algorithm, referred to here as the basic genetic algorithm, `changeptGA` also supports an island model genetic algorithm. The algorithm flowchart is displayed in Figure 2. Building on the basic genetic algorithm, the initialized population is partitioned into islands (sub-populations). On each island, standard genetic algorithm operators – parent selection, crossover, mutation, and sub-population replacement – are executed independently, allowing islands to explore different regions of the search space S in parallel. After a predetermined number of generations, individuals from one island migrate to another, enabling the sharing of genetic material between sub-populations. The least fit individual on each island is replaced by the best fit individual from another randomly picked island. This migration maintains genetic diversity and prevents premature convergence to local optima. By balancing isolated evolution with periodic migration, the island model genetic algorithm enhances overall search efficiency and robustness.

3 Package overview

The `changeptGA` package implements the optimization framework described in Section 2 through two main functions: `cptga`, which applies a basic genetic algorithm, and `cptgaIsl`, which applies an island model genetic algorithm. Both functions take a user-supplied objective function corresponding to $Q(\theta)$ and return an S4 object containing the estimated chromosome, the optimized objective value, and the main genetic algorithm tuning settings. The shared function interface is intended to make it easy to switch between the basic and island model implementations while keeping the same objective function and data inputs. These two primary functions are designed to accommodate the two main optimization settings considered in this paper.

The synopses of genetic algorithm functions `cptga` and `cptgaIsl` are as follows.

```
cptga(ObjFunc, N, prange = NULL, option, ...)
cptgaIsl(ObjFunc, N, prange = NULL, option, ...)
```

Both functions share the leading arguments `ObjFunc`, `N`, and `prange`. The argument `ObjFunc` specifies the user-defined objective function to be minimized, and `N` gives the time series sample

size, which is also used to define the terminal chromosome delimiter $N + 1$. The argument `prange` determines whether model order parameters are included in the chromosome. For changepoint detection only, we set `prange=NULL`, and the task is specified by `option="cp"`. For joint model order selection and changepoint detection, `prange` must be supplied as a list object, and the task is specified by `option="both"`. Each element of `prange` gives the admissible range for one model order parameter, and the number of model order parameters is determined internally as the length of the `prange` list. The remaining arguments control the genetic algorithm search, including the population size, crossover and mutation probabilities, changepoint initialization probability, minimum spacing between changepoints, stopping rules, monitoring, parallel computation, and optional user-defined genetic operators. For `cptgaisl`, additional island model settings specify the number of islands and the migration-based stopping mechanism.

The argument `ObjFunc` must be supplied as an R function that evaluates the fitness of a candidate chromosome and returns a scalar numerical value to be minimized. For example, the `changepointGA` package includes the BIC-based objective functions for the AR(1) model as

```
arma_bic(chromosome, plen = 0, XMat, Xt)
```

where `chromosome` is the candidate chromosome to be evaluated, `plen` denotes the number of model order parameters encoded in the chromosome, and `XMat` and `Xt` are model-specific inputs passed from the main genetic algorithm function call. The genetic algorithm functions determine the value of `plen` from `prange`: `plen=0` when `prange=NULL`, and `plen=length(prange)` when `prange` is supplied as a list. Thus, users should include `plen` as an argument in `ObjFunc` when the objective function needs to separate the model-order component s from the changepoint component τ in the chromosome. When `prange=NULL`, the chromosome is interpreted as $(m, \tau, N + 1)'$; when `prange` is a list, the chromosome is interpreted as $(m, s, \tau, N + 1)'$. Additional named quantities supplied through `...`, such as the observed series, design matrix, or other model-specific constants, are forwarded to `ObjFunc` when their names match the arguments of the objective function. If the returned objective value is non-finite, it is treated as `Inf`, so failed or inadmissible model fits are penalized during the minimization.

To simplify package usage, the task mode is determined by `prange` and `option`, while the objective function argument `plen` records the number of model order parameters encoded in the chromosome. Table 1 summarizes the recommended settings for the two main use cases.

Table 1: Recommended argument settings for the main task modes in `changepointGA`.

Task	plen	prange	option
Changepoint detection only	0	NULL	"cp"
Joint model order selection and changepoint detection	Number of model-order parameters	List of candidate ranges	"both"

The main genetic algorithm tuning parameters control the balance between search breadth, stochastic variation, convergence behavior, and the realism of proposed segmentations. Larger values of population size, `popSize`, and, in the island model, the number of islands, `numIslands`, broaden the search and are often helpful when changepoint detection is combined with model-order selection. The crossover and mutation probabilities, `pcrossover` and `pmutation`, regulate the balance between inheriting information from promising chromosomes and introducing new candidate solutions. The parameter `pchangepoint` influences how readily chromosomes include changepoints, while `minDist` imposes a minimum separation between adjacent changepoints, thereby excluding scientifically implausible short segments. In practice, `minDist` should be chosen using domain knowledge about the shortest meaningful regime length. While larger values of `minDist` can help improve computational efficiency by pruning the search space and reducing spurious detections, the value must remain smaller than the shortest anticipated true regime to ensure that valid changepoint configurations are not excluded from the feasible set.

The function `cptga` is useful when a single evolving population is sufficient, whereas `cptgaisl` is preferable when a more robust and efficient search is desired through multiple sub-populations and migration. In the island model, the total population is partitioned into `numIslands` islands, each of which evolves independently for a prescribed number of generations, `maxgen`, before exchanging highly fit individuals. This strategy often improves exploration of the solution space and can reduce premature convergence.

The package further provides default routines for population initialization, parent selection, crossover, and mutation, while allowing users to replace these components with customized operators

written in either R or Rcpp. This flexibility is important when scientific knowledge suggests problem-specific constraints or when more efficient objective function evaluation is needed. In the simulation studies below, we use the default operators to illustrate the standard workflow. Supplementary Section S1 provides an additional example showing how users may define customized objective functions and operators.

For typical analyses, the workflow is straightforward: specify an objective function, choose whether the task is changepoint detection only or joint order selection, define the associated candidate ranges if needed, and then call either `cptga` or `cptgaisl`. The fitted S4 object can then be examined using `summary` and visualized using `plot`, while `cpt_dist` can be used in simulation settings to compare estimated and true changepoint configurations. Detailed descriptions of all arguments and package options are provided in the package documentation.

4 Simulation studies

This section presents illustrative examples of changepoint detection using time series generated from an $\text{ARMA}(p, q)$. The same framework readily extends to other time series models by specifying an appropriate fitness function and chromosome encoding using the `changepointGA`. For the examples below, let $\{X_t\}_{t=1}^N$ denote a series with m changepoints at unknown locations $\{\tau_1, \dots, \tau_m\}$. These changepoints partition the series into $m + 1$ regimes, where the regimes may differ in mean, trend, or both.

The model considered here is

$$X_t = \kappa_t + \epsilon_t, \quad (3)$$

where ϵ_t follows an $\text{ARMA}(p, q)$ process and κ_t denotes the mean function of X_t . Specifically,

$$\kappa_t = \begin{cases} \beta_0 + \alpha_0 t, & 1 \leq t \leq \tau_1, \\ \beta_0 + \alpha_0 t + \Delta_1 + \alpha_1 t, & \tau_1 + 1 \leq t \leq \tau_2, \\ \vdots & \\ \beta_0 + \alpha_0 t + \Delta_m + \alpha_m t, & \tau_m + 1 \leq t \leq N. \end{cases} \quad (4)$$

Here, β_0 and α_0 are the baseline intercept and trend parameters, respectively, while $\Delta_1, \dots, \Delta_m$ and $\alpha_1, \dots, \alpha_m$ denote the segment-specific level and trend changes associated with the changepoints.

When $m > 1$, the goal is to detect multiple changepoints. We first consider settings without model-order selection to demonstrate the use of `changepointGA` and to compare its detection performance with the general-purpose `GA` package (Scrucca, 2013), with particular emphasis on computational efficiency. We then present an example in which changepoint detection is performed jointly with model-order selection, a feature not available in `GA`. In these experiments, the classical BIC is used as an illustrative model-selection criterion. However, the resulting accuracy may depend on the criterion chosen by the user, such as mBIC, MDL, or related alternatives. An additional AMOC example ($m = 1$) is provided in the supplementary materials, where we also illustrate how users can define customized objective functions and genetic operators.

4.1 $\text{ARMA}(p, q)$ orders known

By default, `changepointGA` targets multiple changepoint detection. Methods for this task are commonly grouped into two main classes: (i) binary segmentation based on AMOC procedures, and (ii) direct model selection approaches that jointly fit all subsegments. When model orders are known, the latter treats the full series simultaneously by optimizing an objective function that comprises the negative log-likelihood and a penalty that balances goodness of fit and model complexity. For our illustrative example governed by Equation (3), the BIC is defined as:

$$Q(\theta) = -2\log(L(\theta)) + k\log(N), \quad (5)$$

where k denotes the number of model parameters. The term $-2\log(L(\theta))$ measures the lack of fit for a model with m changepoints in τ , while the second term serves as a penalty to prevent overfitting. A smaller BIC indicates a better model fit. Minimizing $Q(\theta)$ selects the best-fitting model, and the resulting changepoint configuration provides the estimated locations of the changepoints.

Assuming $\{\epsilon_t\}$ in Equation (3) follows an $\text{AR}(1)$ process ($p = 1, q = 0$),

$$\epsilon_t = \phi\epsilon_{t-1} + e_t, \quad (6)$$

where $|\phi| < 1$ is the autoregressive parameter and $\{e_t\}$ denotes a white noise process with mean zero

and variance σ^2 . The corresponding BIC can be computed using the `arma_bic` function from the `changepointGA` package.

In designing the objective function, the chromosome argument holds the individual changepoint configuration C as described in Equation (2). First, we extract the number and locations of the changepoints from the chromosome for fitness calculation. To inform the package that the model order selection is not considered here, it is essential to set `plen = 0`, indicating that the vector including model orders has a length of zero. The time series data in X_t are input into the objective function and will be included as an additional argument passed to both the `cptga` and `cptgaisl` functions during execution.

To illustrate the detection of changepoints with our package, we simulated a time series following the models specified in Equations (3), (4), and (6) with $N = 1000$, $\phi = 0.5$, and $\sigma^2 = 1$. For simplicity, all simulations below set $\alpha_i = 0$ for $i = 0, 1, \dots, m$, so that the regimes differ only in their mean levels. Trend shifts are illustrated using the real-data application in the next section.

Two changepoints

Two changepoints were introduced at $\tau_1 = 250$ and $\tau_2 = 750$, resulting in three segments with means $\beta_0 = 0.5$, $\beta_0 + \Delta_1 = 0.5 + 2 = 2.5$ and $\beta_0 + \Delta_2 = 0.5 + (-2) = -1.5$, respectively. The series can be generated using the function `ts_sim`.

```
N <- 1000
XMatT <- matrix(1, nrow = N, ncol = 1)
Xt <- ts_sim(Ts = N, beta = 0.5, XMat = XMatT, sigma = 1, phi = 0.5, theta = NULL,
  Delta = c(2, -2), CpLoc = c(250, 750), seed = 12345)
```

The introduced changepoints can be examined in Figure 3. The segments are marked by vertical blue dashed lines and the segment sample means are marked by horizontal red dashed lines.

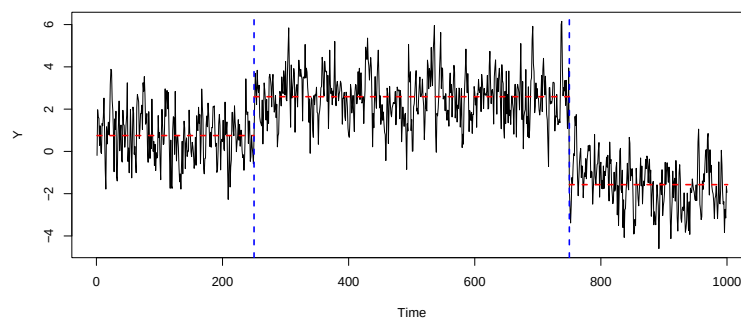


Figure 3: A simulated AR(1) time series plot with the changepoint segmentation (blue dashed lines) and the corresponding sample means (red dashed lines).

The basic genetic algorithm with the `arma_bic` objective function and default genetic operators can be implemented as follows. During the search, setting `monitoring = TRUE` displays the best-fitting changepoint configuration and its corresponding objective function value at each generation. Upon completion, the algorithm returns an S4 object, `res_cptga`, containing details of the genetic algorithm hyperparameters and the estimated number and locations of changepoints. A summary can be obtained with the `summary()` function. The objective function value $Q(\hat{\theta})$ and the associated changepoint configuration $\hat{C}$ for the best-fitting model can also be extracted, as illustrated below.

```
res_cptga <- cptga(
  ObjFunc = arma_bic, N = N, popSize = 100, pcrossover = 0.95,
  pmutation = 0.3, pchangepoint = 10 / N, seed = 1234,
  XMat = XMatT, Xt = Xt
)
summary(res_cptga)

#> #####
#> #           Changepoint Detection via GA           #
#> #####
#> Settings:
#> Population size           = 100
```

```

#> Number of generations = 5271
#> Crossover probability = 0.95
#> Mutation probability = 0.3
#> Changepoint probability = 0.01
#> minDist = 1
#> Task mode = cp
#> Parallel Usage = FALSE
#> Seed = 1234
#>
#> ##### GA results #####
#> Optimal Fitness value = 68.54954
#> Optimal Solution:
#>   Number of Changepoints = 2
#>   Changepoints Locations = 250 750

fit_cptga <- res_cptga@overbestfit
fit_cptga

#> [1] 68.54954

tau_cptga <- res_cptga@overbestchrom
tau_cptga

#> [1] 2 250 750 1001

```

In this example, the basic genetic algorithm detected 2 changepoints, $\hat{\tau} = \{250, 750\}$, with the chromosome ending flag of 1001. The minimized BIC is 68.55.

For the island model genetic algorithm, we maintain an initial population size of 100 and set `numIslands = 5`, yielding 20 individuals per island. To align the stopping criterion with `cptga` – where the algorithm terminates once the overall best-fitting value remains unchanged for 5000 consecutive generations – we specify `maxgen = 50` and `maxconv = 100` for `cptgaisl`. The same genetic operators are applied to generate new individuals at each generation. The estimated changepoint configuration and fitness value can be extracted in the same manner.

```

res_cptgaisl <- cptgaisl(
  ObjFunc = arima_bic, N = N, popSize = 100, numIslands = 5,
  pcrossover = 0.95, pmutation = 0.3, pchangeoint = 10 / N,
  seed = 1234, XMat = XMatT, Xt = Xt
)
summary(res_cptgaisl)

#> #####
#> # Changepoint Detection via Island Model GA #
#> #####
#> Settings:
#> Population size = 100
#> Number of Island = 5
#> Island size = 20
#> Number of generations = 5150
#> Number of migrations = 103
#> Crossover probability = 0.95
#> Mutation probability = 0.3
#> Changepoint probability = 0.01
#> minDist = 1
#> Task mode = cp
#> Parallel Usage = FALSE
#> Seed = 1234
#>
#> ##### Island Model GA results #####
#> Optimal Fitness value = 68.54954
#> Optimal Solution:
#>   Number of Changepoints = 2
#>   Changepoints Locations = 250 750

```

```
fit_cptgaisl <- res_cptgaisl@overbestfit
fit_cptgaisl
```

```
#> [1] 68.54954
```

```
tau_cptgaisl <- res_cptgaisl@overbestchrom
tau_cptgaisl
```

```
#> [1] 2 250 750 1001
```

The island model genetic algorithm detected 2 changepoints, $\hat{\tau} = \{250, 750\}$, with the chromosome ending flag of 1001. The minimized BIC is 68.55, identical to that obtained with the basic genetic algorithm in this run. As genetic algorithms are stochastic search methods, results may vary across executions.

By comparing the changepoints estimated by the basic genetic algorithm and the island model genetic algorithm with the true values (250, 750) used in the simulation, discrepancies in both number and locations can be quantified using the distance metric implemented in `cpt_dist`.

```
m_cptga <- tau_cptga[1]
tau_cptga <- tau_cptga[2:(1 + m_cptga)]
dist_cptga <- cpt_dist(tau1 = tau_cptga, tau2 = c(250, 750), N = N)
dist_cptga
```

```
#> [1] 0
```

```
m_cptgaisl <- tau_cptgaisl[1]
tau_cptgaisl <- tau_cptgaisl[2:(1 + m_cptgaisl)]
dist_cptgaisl <- cpt_dist(tau1 = tau_cptgaisl, tau2 = c(250, 750), N = N)
dist_cptgaisl
```

```
#> [1] 0
```

Both `cptga` and `cptgaisl` produced changepoint configurations closely matching the ground truth. No substantial discrepancies were observed between the estimated and true changepoint locations used in data generation, yielding `cpt_dist` values near zero.

We next compare the robustness and computational efficiency of `changepointGA` and `GA`. Using the same simulation setup as above, we generated 1000 AR(1) time series and applied `cptga` and `cptgaisl` from `changepointGA`, as well as `ga` and `gaisl` from `GA`, to each simulated series. Here, `cptga` and `ga` implement the basic genetic algorithm, while `cptgaisl` and `gaisl` implement the island model genetic algorithm.

Five scenarios were considered, differing by package, population size, and stopping criterion, as summarized in Table 2. Each scenario is labeled in the format Package–Population size–Stopping criteria, where “GA” denotes the `GA` package and “CG” denotes the `changepointGA` package. Population sizes are categorized as “S” (small, 100 individuals), “M” (medium, 400 individuals), and “L” (large, 800 individuals), while stopping criteria are indicated as “1K” (1000) or “5K” (5000), representing the maximum number of consecutive generations or migrations without fitness improvement before termination. For example, CG-S-1K corresponds to `changepointGA` with a small population of 100 and a stopping criterion of 1000.

Table 2: Different hyperparameter settings for the basic and island model genetic algorithm implementations in the `GA` and `changepointGA` packages.

Scenarios	Population size	Stopping criteria	Package
GA-S-1K	100	1000	GA
CG-S-1K	100	1000	changepointGA
CG-S-5K	100	5000	changepointGA
CG-M-5K	400	5000	changepointGA
CG-L-5K	800	5000	changepointGA

To ensure comparability, key hyperparameters were aligned across the basic and island model genetic algorithm runs in both packages. In the island model genetic algorithm, populations were

evenly distributed across islands: 100 individuals across five islands, 400 across 20 islands, and 800 across 40 islands, maintaining a fixed island size of 20. Migration occurred every 50 generations. For scenarios with a 1K stopping criterion, convergence was assessed after 20 consecutive migrations without fitness improvement; for 5K, the maximum number of consecutive migrations was increased to 100. As the scenario progressed from CG-S-1K to CG-L-5K, both population size and the stopping criterion increased. These more intensive settings were expected to yield solutions closer to the global optimum by providing a larger search budget and improved convergence behavior.

The fitness function `ARIMA_BIC_GA`, provided in the supplementary materials, closely mirrors `arma_bic` from `changePointGA`, with the primary distinction being the initial chromosome structure, which was adapted to match the input format required by `GA`. In addition, three values of the minimum changepoint spacing parameter were examined, with `minDist=1` used as the default in both packages. Since our `changePointGA` package allows variation in minimum segment length – a unique feature of `changePointGA` – larger values (`minDist = 3, 5`) were also tested to investigate the impact of imposing a minimum segment length on changepoint detection.

The performance of changepoint detection was evaluated in terms of detection accuracy, computational efficiency, and optimized fitness function values. Figure 4 illustrates detection accuracy by plotting the average distance, as evaluated by the `cpt_dist` function, between the estimated changepoint configuration and the true configuration $C = (2, 250, 750, 1001)'$ across 1000 simulations. For each of the four scenarios using `changePointGA`, results are reported for three minimum changepoint spacing values (`minDist = 1, 3, 5`), whereas for the GA-S-1K scenario only `minDist = 1` is reported (since this option is not available in `GA`). Across the five scenarios, both the basic and island model genetic algorithm implementations in `changePointGA` achieved smaller average distances than GA-S-1K in `GA`, indicating higher detection accuracy. In general, as `minDist` increases, `changePointGA` produced slightly smaller average distances, with the best performance observed under the most intensive search setting (CG-L-5K) with `minDist = 5`.

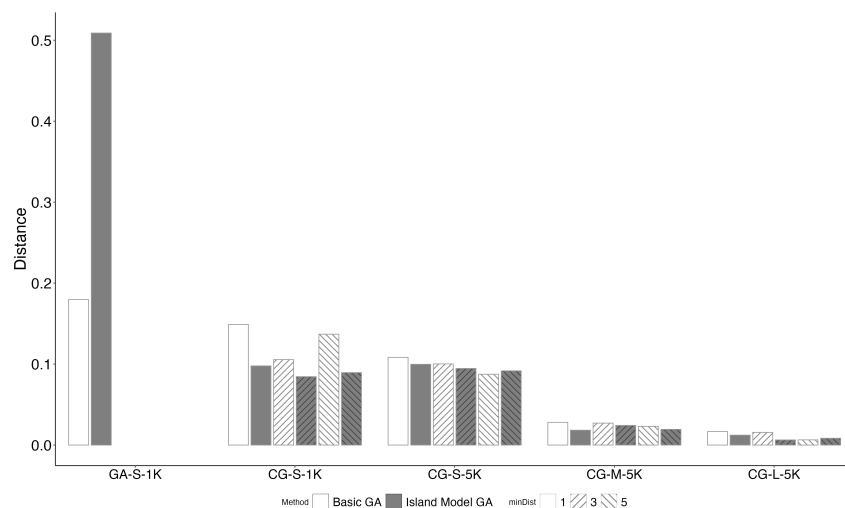


Figure 4: Barplots showing the average distance between true and estimated changepoint locations for the basic genetic algorithm (Basic GA) and the island model genetic algorithm (Island Model GA) under different simulation scenarios.

Table 3 summarizes computational efficiency by comparing average execution times. By narrowing the search space through the chromosome design in Equation (2), the `changePointGA` implementations substantially reduced computation time relative to `GA`. For instance, with `minDist = 1`, the average runtime for the GA-S-1K scenario was approximately 3500 seconds, compared with about one second for CG-S-1K – an almost 3500-fold reduction. Even with larger population sizes and more stringent stopping criteria, such as CG-L-5K, the average runtime remained about 535 times shorter for the basic genetic algorithm and 161 times shorter for the island model genetic algorithm relative to GA-S-1K. For `minDist = 3` and `5`, the search space was further reduced by excluding unrealistic changepoint configurations, dropping the CG-S-1K average runtime to 0.30 seconds for the basic genetic algorithm and 0.70 seconds for the island model genetic algorithm.

Across `changePointGA` scenarios, the island model genetic algorithm consistently improved detection performance relative to the basic genetic algorithm (Figure 4), with only a small runtime increase (Table 3). As expected, increasing population size and tightening the stopping criterion raised execution time but decreased average distance, thereby improving detection accuracy.

Table 3: Average computation time (seconds) from 1000 simulations of the basic genetic algorithm (Basic GA) and island model genetic algorithm (Island Model GA) across five scenarios and three minimum changepoint spacing values.

minDist	Basic GA					Island Model GA				
	GA-S-1K	CG-S-1K	CG-S-5K	CG-M-5K	CG-L-5K	GA-S-1K	CG-S-1K	CG-S-5K	CG-M-5K	CG-L-5K
1	3512.69	0.98	1.92	3.78	6.56	3428.14	1.25	5.07	19.98	21.23
3	—	0.30	0.83	1.62	2.62	—	0.70	2.85	11.36	11.92
5	—	0.30	0.84	1.58	2.63	—	0.70	2.90	11.51	12.08

To further evaluate changepoint detection performance, the empirical frequency distributions of the number of estimated changepoints are reported in Table 4. For both the basic and island model genetic algorithm implementations in `changepointGA`, the true value of two changepoints was selected more frequently as the search budget increased. The percentage of simulations (out of 1000) yielding the correct number of changepoints with `changepointGA` was generally higher than that for the GA-S-1K scenario, except for CG-S-1K with $\text{minDist} = 1$. Although GA-S-1K appears more concentrated around $m = 2$ than CG-S-1K for the basic genetic algorithm, the overall distribution for GA-S-1K is more dispersed and yields a larger average changepoint-location distance, as shown in Figure 4. Under a small search budget, the basic genetic algorithm implementation in `changepointGA` occasionally selected near-miss models, such as $m = 3$, more often than GA-S-1K. This behavior may reflect the rapid convergence of the compact integer encoding to nearby configurations when the search budget is limited. The island model implementation mitigated this behavior, with `changepointGA` selecting the true value $m = 2$ more consistently.

Moreover, the results in Table 4 show that the island model genetic algorithm in `changepointGA` consistently improved the accuracy of estimating the number of changepoints relative to the corresponding basic genetic algorithm implementation across values of minDist and most search settings. Increasing minDist from 1 to 3 or 5 generally maintained or improved the frequency with which the true number of changepoints, $m = 2$, was selected for both the basic and island model genetic algorithms, although the improvement was not strictly monotone in all cases. The effect was most apparent for CG-S-1K under the basic genetic algorithm implementation, where the accuracy increased from 86.5% at $\text{minDist}=1$ to 91.0% at $\text{minDist}=3$, before decreasing to 88.0% at $\text{minDist}=5$. Thus, for this smaller-search setting, minDist produced an inverted U-shaped accuracy pattern, or equivalently, a roughly U-shaped error pattern. For larger search settings, such as CG-M-5K and CG-L-5K, the correct number of changepoints was selected with high frequency regardless of minDist , suggesting that the minimum segment length constraint has a more modest effect when the search budget is sufficiently large. Overall, these results suggest that minDist is most useful as a regularization device when the search budget is limited, helping to reduce the tendency to select unrealistic nearby changepoint configurations. In real applications, we recommend choosing minDist based on prior knowledge of the shortest meaningful regime length. When such information is unavailable, a sensitivity analysis over several small values may be useful.

Furthermore, we investigate whether `changepointGA` or GA attains the minimum BIC value more frequently under the fair comparison setting with $\text{minDist} = 1$. Figure 5 presents stacked barplots comparing the optimized fitness function values between GA-S-1K and each scenario using `changepointGA` for both the basic genetic algorithm in panel A and the island model genetic algorithm in panel B. For each simulation run, there are three possible outcomes: if `changepointGA` achieves a smaller BIC, the run is categorized as “`changepointGA` better”; if both `changepointGA` and GA yield identical BICs, the outcome is labeled “Equal”; and if GA yields a smaller BIC, the run is categorized as “GA better”.

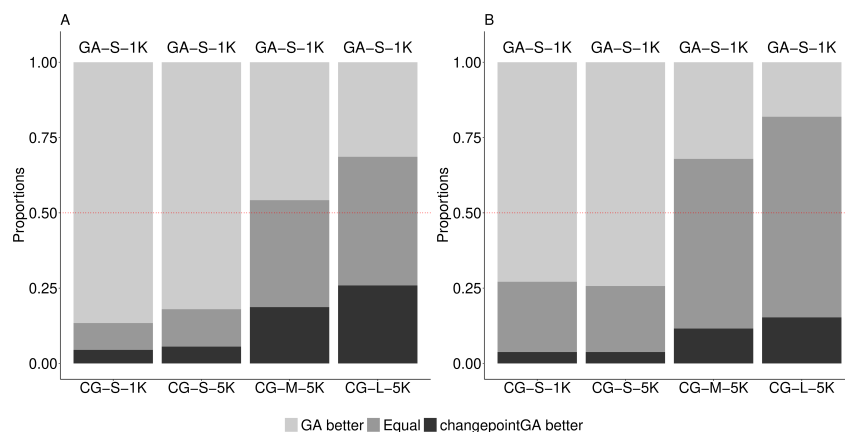
It is evident from Figure 5 that `changepointGA` sacrifices a degree of optimization accuracy in exchange for faster runtime. For example, CG-L-5K and GA-S-1K agree most frequently, with CG-L-5K attaining the lowest BIC more often in both the basic genetic algorithm and island model genetic algorithm implementations among scenarios using `changepointGA`. Although the population size increases from 100 (GA-S-1K) to 800 (CG-L-5K) with more stringent stopping criteria, `changepointGA` achieves comparable optimization performance, with a much shorter average distance and a much higher rate of correctly detecting two changepoints (true $m = 2$). Most importantly, it runs over 150 times faster.

Six changepoints

To further assess robustness, we conducted an additional simulation study under a more challenging multiple changepoint setting. The ARMA orders were treated as known, with $p = 1$ and $q = 0$. In all cases, the series length was fixed at $T = 1000$, and six true changepoint locations were set at 200, 400, 500, 700, 750, and 850. The corresponding mean shift magnitudes were specified as

Table 4: Estimated numbers of changepoints (percentages across 1000 simulations) for the basic genetic algorithm (Basic GA) and the island model genetic algorithm (Island Model GA) across five scenarios.

minDist	m	Basic GA					Island Model GA				
		GA-S-1K	CG-S-1K	CG-S-5K	CG-M-5K	CG-L-5K	GA-S-1K	CG-S-1K	CG-S-5K	CG-M-5K	CG-L-5K
1	0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	1	0.0	0.0	0.3	0.0	0.0	0.0	0.0	0.1	0.0	0.0
	2	89.8	86.5	90.1	97.6	98.7	75.6	91.4	91.3	98.6	99.1
	3	4.7	12.9	9.7	2.4	1.3	3.9	8.2	8.2	1.4	0.9
	4	4.3	0.6	0.2	0.0	0.0	17.2	0.4	0.5	0.0	0.0
	5	0.6	0.0	0.0	0.0	0.0	1.5	0.0	0.0	0.0	0.0
	6	0.5	0.0	0.0	0.0	0.0	1.6	0.0	0.0	0.0	0.0
	>7	0.0	0.0	0.0	0.0	0.0	0.1	0.0	0.0	0.0	0.0
3	0	—	0.0	0.0	0.0	0.0	—	0.0	0.0	0.0	0.0
	1	—	0.0	0.0	0.0	0.0	—	0.0	0.0	0.0	0.0
	2	—	91.0	91.2	97.7	98.8	—	92.4	91.6	98.0	99.7
	3	—	8.2	8.3	2.3	1.2	—	7.5	8.1	2.0	0.3
	4	—	0.8	0.5	0.0	0.0	—	0.1	0.3	0.0	0.0
	5	—	0.0	0.0	0.0	0.0	—	0.0	0.0	0.0	0.0
	6	—	0.0	0.0	0.0	0.0	—	0.0	0.0	0.0	0.0
	>7	—	0.0	0.0	0.0	0.0	—	0.0	0.0	0.0	0.0
5	0	—	0.0	0.0	0.0	0.0	—	0.0	0.0	0.0	0.0
	1	—	0.0	0.0	0.0	0.0	—	0.0	0.0	0.0	0.0
	2	—	88.0	92.2	98.1	99.7	—	92.3	92.0	98.5	99.5
	3	—	11.1	7.6	1.9	0.3	—	7.2	7.6	1.5	0.5
	4	—	0.9	0.2	0.0	0.0	—	0.5	0.4	0.0	0.0
	5	—	0.0	0.0	0.0	0.0	—	0.0	0.0	0.0	0.0
	6	—	0.0	0.0	0.0	0.0	—	0.0	0.0	0.0	0.0
	>7	—	0.0	0.0	0.0	0.0	—	0.0	0.0	0.0	0.0

**Figure 5:** Stacked barplots comparing optimized BIC values between the GA-S-1K scenario from the GA package and the CG-S-1K, CG-S-5K, CG-M-5K, and CG-L-5K scenarios from changepointGA, using the basic genetic algorithm (A) and island model genetic algorithm (B).

$a \times (1, 2, 1, 0, 1, 2)$, where $a \in \{1, 2, 3\}$ controls the overall signal strength. The error process was generated from the same AR(1) model used above, with $\phi = 0.5$ and $\sigma^2 = 1$.

For each signal setting, 1000 simulated series were generated. Both `cptga` and `cptgaisl` were run using sequential computation (`parallel = FALSE`) and parallel computation (`parallel = TRUE` with `nCore = 10`). This design allows us to assess how detection accuracy varies with signal strength, with particular attention to whether the nearby changepoints at 700 and 750 are successfully identified, and how much runtime can be reduced through parallelization. The estimated changepoint locations from the basic genetic algorithm and island model genetic algorithm implementations are displayed in Figure 6.

The histograms in Figure 6 show that changepoint detection improves as the signal strength a increases. For $a=1$, both methods most clearly identify the larger shifts, whereas the smaller and nearby shifts, especially those around 700 and 750, are detected less consistently. For $a=2$ and $a=3$, the estimated changepoint locations become more concentrated around the true changepoints, with the island model genetic algorithm generally producing sharper peaks than the basic genetic algorithm. The sequential

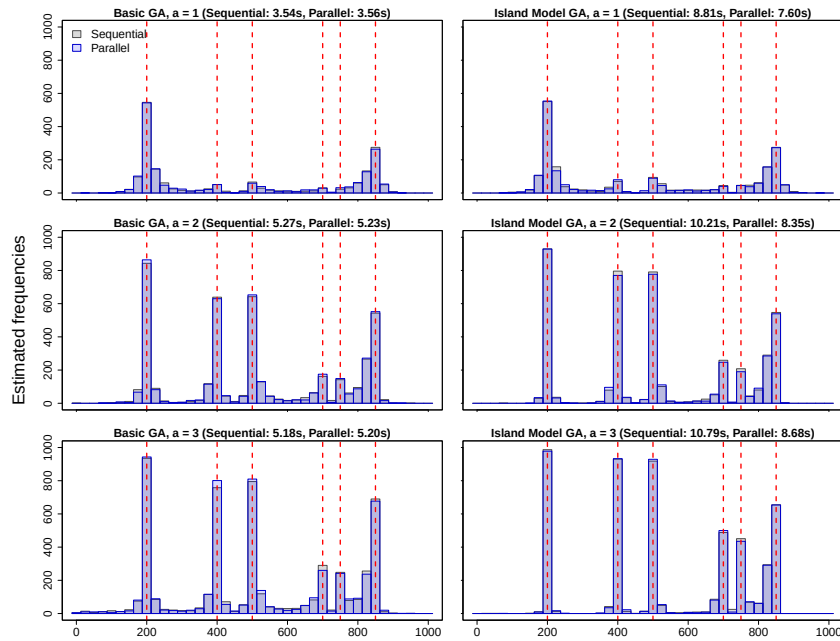


Figure 6: Histograms of the estimated changepoint locations with known ARMA orders, using the basic genetic algorithm (Basic GA in the left column) and the island model genetic algorithm (Island Model GA in the right column). Red dashed lines indicate the true changepoint positions; gray and blue histograms show sequential and parallel results, respectively.

and parallel implementations yield nearly identical detection patterns, indicating that parallelization primarily affects runtime rather than estimation accuracy. The main benefit of parallelization is computational: the reported runtimes show that parallel computation reduces runtime for the island model genetic algorithm, whereas the runtime savings are small for the basic genetic algorithm. Because parallelization is implemented primarily in fitness evaluations, its computational advantage is expected to be greater when the population size is large or when the objective function is expensive to evaluate.

No changepoints

Lastly, to assess false positive behavior, we considered a no-changepoint setting with series length $T = 1000$. The data were generated from a stationary AR(1) process with $\phi = 0.5$ and $\sigma^2 = 1$. No mean or trend shifts were introduced, so the true number of changepoints is $m = 0$. This setting provides a baseline for evaluating whether the `changepointGA` spuriously detects changepoints in a stationary time series. Among 1000 simulated series, the basic genetic algorithm correctly detected no changepoints in 99.7% of cases under both sequential and parallel computation. The island model genetic algorithm correctly detected no changepoints in 99.3% and 99.7% of cases under sequential and parallel computation, respectively.

4.2 ARMA(p, q) orders unknown

In practical applications, the orders of an ARMA(p, q) model are typically unknown. A more robust approach is to jointly estimate the model orders, the number of changepoints, and their locations. Even without prespecifying p and q , the BIC criterion in Equation (5) can still be applied to simultaneously detect multiple changepoints and select appropriate model orders.

In this subsection, ϵ_t in Equation (3) is assumed to follow an ARMA(1,1) process:

$$\epsilon_t = \phi_1 \epsilon_{t-1} + e_t + \theta_1 e_{t-1}, \quad (7)$$

where $|\phi_1| < 1$ is the autoregressive parameter, $|\theta_1| < 1$ is the moving average parameter, and $\{e_t\}$ denotes a white noise process with mean zero and variance σ^2 . In the following illustration, the arguments ϕ and θ correspond to $\phi_1 = 0.5$ and $\theta_1 = 0.8$, respectively. Data are generated from the chromosome $C = (2, 1, 1, 250, 750, 1001)'$, which encodes an ARMA(1,1) process with $m = 2$

changepoints at times 250 and 750. All other parameters follow those in the previous subsection.

```
N <- 1000
XMatT <- matrix(1, nrow = N, ncol = 1)
Xt <- ts_sim(Ts=N, beta = 0.5, XMat = XMatT, sigma = 1, phi = 0.5, theta = 0.8,
  Delta = c(2, -2), CpLoc = c(250, 750), seed = 123456)
```

When model order selection is required, the argument `plen` in the objective function `arma_bic_order_pq` must be specified to inform the `changepointGA` package. For an ARMA(1,1) model, setting `plen=2` indicates that two model orders are to be selected. Both the time series data `Xt` and the regression design matrix `XMat` are included as additional arguments. Inside `arma_bic_order_pq`, ARMA model fitting is performed using the standard R function `arma`. The penalty term $m\log(N)$ in the BIC criterion is intended to reduce the tendency to overestimate the number of changepoints, particularly for time series in which unmodeled positive serial correlation may otherwise be captured by spurious changepoints.

```
arma_bic_order_pq(chromosome, plen = 2, XMat, Xt)
```

In `cptga` and `cptgaisl` functions, the permissible range of each order parameter is defined through a list object `prange`, which contains integer ranges for the parameters. For example, in the code snippet below, `prange` specifies that both the AR and MA orders may take values from $\{0, 1, 2, 3\}$. This list is then passed to the `cptga` and `cptgaisl`, where the number of model order parameters equals the length of `prange`. Additionally, for order selection, set `option="both"` in the call. To address the increased computational complexity of optimization, the population size is set to `popSize = 800` in `cptga`, and to `popSize = 800` with `numIslands = 10` in `cptgaisl`. With `minDist = 3`, the objective function `arma_bic_order_pq` is then minimized using both the basic genetic algorithm and island model genetic algorithm, as illustrated below.

```
prange <- list(ar = c(0, 3), ma = c(0, 3))
res_cptga <- cptga(
  ObjFunc = arma_bic_order_pq, N = N, prange = prange,
  popSize = 800, pcrossover = 0.95, pmutation = 0.3,
  pchangeoint = 10 / N, option = "both", minDist = 3,
  XMat = XMatT, Xt = Xt, seed = 123456
)
```

```
summary(res_cptga)
```

```
#> #####
#> #           Changepoint Detection via GA           #
#> #####
#> Settings:
#> Population size           = 800
#> Number of generations     = 13508
#> Crossover probability     = 0.95
#> Mutation probability      = 0.3
#> Changepoint probability   = 0.01
#> minDist                   = 3
#> Task mode                 = both
#> Parallel Usage           = FALSE
#> Seed                      = 123456
#>
#> ##### GA results #####
#> Optimal Fitness value = 2877.265
#> Optimal Solution:
#>   Number of Changepoints = 2
#>   Model hyperparameters:
#>     ar = 1
#>     ma = 1
#>   Changepoints Locations = 265 749
```

```
res_cptgaisl <- cptgaisl(
  ObjFunc = arma_bic_order_pq, N = N, prange = prange,
```

```

popSize = 800, numIslands = 10,
pcrossover = 0.95, pmutation = 0.3,
pchangePoint = 10 / N, option = "both", minDist = 3,
XMat = XMatT, Xt = Xt, seed = 123456
)

summary(res_cptgaisl)

#> #####
#> # Changepoint Detection via Island Model GA #
#> #####
#> Settings:
#> Population size      = 800
#> Number of Island     = 10
#> Island size          = 80
#> Number of generations = 5650
#> Number of migrations = 113
#> Crossover probability = 0.95
#> Mutation probability  = 0.3
#> Changepoint probability = 0.01
#> minDist              = 3
#> Task mode             = both
#> Parallel Usage        = FALSE
#> Seed                  = 123456
#>
#> ##### Island Model GA results #####
#> Optimal Fitness value = 2870.097
#> Optimal Solution:
#>   Number of Changepoints = 2
#>   Model hyperparameters:
#>     ar = 1
#>     ma = 1
#>   Changepoints Locations = 244 749

```

Both the basic genetic algorithm and the island model genetic algorithm successfully estimated the AR and MA orders ($p = 1$ and $q = 1$) and identified changepoints near their true locations. The island model genetic algorithm required fewer generations than the basic genetic algorithm, highlighting its computational advantage. Since genetic algorithms are stochastic search algorithms (as mentioned earlier), results may vary slightly across simulations. The distance metric function `cpt_dist` can still be used to measure differences between changepoint configurations, as described in the previous subsection.

Table 5: Estimated number of changepoints and ARMA(p, q) orders (percentages across 1000 simulations) for the basic genetic algorithm (Basic GA) and the island model genetic algorithm (Island Model GA) in `changepointGA`.

m	Number of changepoints		Orders of ARMA models					
	Basic GA	Island Model GA	(p, q)	Basic GA	Island Model GA	(p, q)	Basic GA	Island Model GA
0	0.6	0.7	(0, 0)	0.0	0.0	(2, 0)	0.0	0.0
1	6.5	3.9	(0, 1)	0.0	0.0	(2, 1)	9.7	6.3
2	83.5	91.7	(0, 2)	0.0	0.0	(2, 2)	13.8	4.5
3	8.6	3.7	(0, 3)	0.9	0.4	(2, 3)	0.9	0.1
4	0.8	0.0	(1, 0)	0.0	0.0	(3, 0)	0.0	0.0
5	0.0	0.0	(1, 1)	59.5	80.7	(3, 1)	2.8	0.0
6	0.0	0.0	(1, 2)	9.2	7.8	(3, 2)	0.4	0.1
>6	0.0	0.0	(1, 3)	2.6	0.1	(3, 3)	0.2	0.0

In a simulation study of 1000 runs using the same setup as above, Table 5 and Figure 7 summarize the results for both the basic genetic algorithm and the island model genetic algorithm implementations in `changepointGA`. A direct comparison with the `GA` package is not available in this setting, because `GA` does not provide joint estimation of ARMA orders and changepoint locations. Table 5 reports the empirical frequency distributions of the estimated number of changepoints and ARMA(p, q) orders. The correct number of changepoints was identified in 83.5% of runs with the basic genetic algorithm and 91.7% with the island model genetic algorithm. Across all 16 possible (p, q) combinations from $\{0, 1, 2, 3\}$, 59.5% of the simulations correctly recovered the true ARMA model orders ($p = 1, q = 1$)

for the basic genetic algorithm, and 80.7% correctly recovered the true orders for the island model genetic algorithm.

Additionally, Figure 7 presents histograms of the estimated changepoint locations for both the basic genetic algorithm and the island model genetic algorithm, showing the total number of times a changepoint was detected at time t for $1 \leq t < N$. The modes of the histograms are concentrated around the true changepoint times ($t = 250$ and $t = 750$). Detection at $t = 750$ is more frequent than at $t = 250$, reflecting the larger magnitude of change at $t = 750$. The average distance between the estimated and true changepoint configurations was 0.185 for the basic genetic algorithm and 0.092 for the island model genetic algorithm, showing improved changepoint-location recovery under the island model. This is consistent with the island model genetic algorithm's higher recovery rates for both the correct number of changepoints and the true ARMA(1, 1) order. Overall, the `changepointGA` package provides robust joint estimation of both changepoints and ARMA model orders.

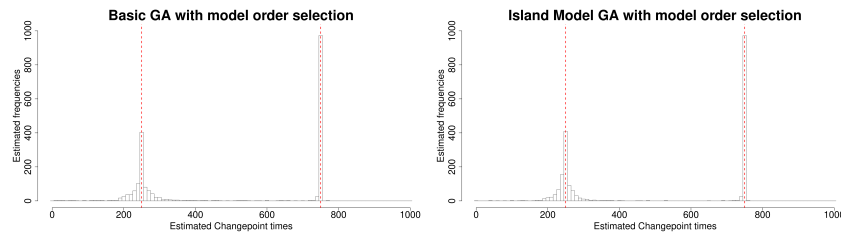


Figure 7: Histograms of estimated changepoint times for the basic genetic algorithm (Basic GA) and the island model genetic algorithm (Island Model GA) in `changepointGA`.

5 Applications

5.1 Glioblastoma aCGH data

Glioblastoma multiforme (GBM) can induce chromosomal aberrations during DNA replication. Array comparative genomic hybridization (aCGH) experiments analyze these aberrations through fluorescence intensity ratios, which reflect DNA copy number variations: ratios greater than one indicate chromosomal gains, whereas ratios less than one indicate deletions. Changepoint detection provides an effective approach to gene sequence segmentation, improving the accuracy of genomic alteration detection and supporting advances in disease screening and treatment. The aCGH data used here replicate those reported in Killick and Eckley (2014) and Lai et al. (2005).

Following Killick and Eckley (2014) and Lai et al. (2005), we considered the piecewise constant mean shift model described in Equation (3), with ϵ_t assumed to follow an IID normal distribution and all trend parameters in Equation (4) set to zero. The BIC was used to construct the objective function `lm_search_bic`, provided in the supplementary materials. To avoid redundancy, only `cptgaisl` was employed to minimize this objective function for segmentation of the aCGH data.

```
load("data/arrayaCGH.rda")
Xt <- arrayaCGH[, 5]
N <- length(Xt)
XMat <- matrix(1, nrow = N, ncol = 1)
res_cptgaisl <- cptgaisl(
  ObjFunc = lm_search_bic, N = N, popSize = 400, numIslands = 10,
  pcrossover = 0.95, pmutation = 0.3, pchangepoint = 10/N,
  XMat = XMat, Xt = Xt, seed = 123456)
summary(res_cptgaisl)

#> #####
#> # Changepoint Detection via Island Model GA #
#> #####
#> Settings:
#> Population size      = 400
#> Number of Island    = 10
#> Island size         = 40
#> Number of generations = 6600
#> Number of migrations = 132
#> Crossover probability = 0.95
```

```
#> Mutation probability      = 0.3
#> Changepoint probability = 0.05181347
#> minDist                  = 1
#> Task mode                = cp
#> Parallel Usage           = FALSE
#> Seed                    = 123456
#>
#> ##### Island Model GA results #####
#> Optimal Fitness value = 391.2571
#> Optimal Solution:
#>   Number of Changepoints = 6
#>   Changepoints Locations = 81 85 89 96 123 133
```

There are $\hat{m} = 6$ detected changepoints in the sequence, indicating DNA copy number variations at $\hat{\tau} = \{81, 85, 89, 96, 123, 133\}$. These results closely align with those obtained from other segmentation methods, such as CGHseg (Killick and Eckley, 2014) and PELT (Lai et al., 2005). Our implementation successfully recovered all changepoints identified by these methods and additionally detected {89} as a changepoint. The additional changepoint was also detected by the wavelet (Hsu et al., 2005), GLAD (Hupé et al., 2004), CLAC (Wang et al., 2005), and Quantreg (Eilers and de Menezes, 2005) segmentation approaches implemented in Killick and Eckley (2014). The sequence array profile with the detected changepoints is shown in Figure 8.

```
plot(res_cptgaisl, data = Xt, XAxisLab = "Index", YAxisLab = "Intensity Ratios")
```

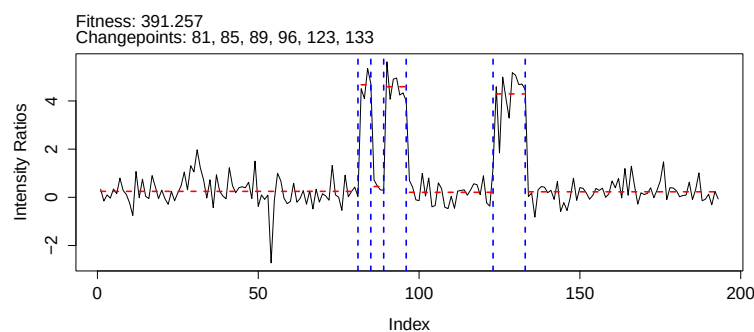


Figure 8: A changepoint analysis for chromosomal segment detection in Glioblastoma aCGH data.

5.2 Atlanta airport temperature series

In this example, we demonstrate the importance of simultaneously selecting time series model orders and detecting changepoints. The annual mean temperature data from 1879 to 2013 at Atlanta, Georgia's Hartsfield International Airport weather station, available in Lund et al. (2023), were reanalyzed using our `changepointGA` package.

```
load("data/AtlantaTemperature.rda")
Xt <- AtlantaTemperature[, 2]
N <- length(Xt)
```

Three piecewise models are fitted to the Atlanta series. The first is the constant mean shift model, in which all trend parameters satisfy $\alpha_i = 0$ for $i = 0, 1, \dots, m$ in Equation (4). The second is the fixed common-trend mean shift model, in which $\alpha_i = 0$ for $i = 1, \dots, m$, so that all regimes share the baseline trend. The third is the trend shift model specified in Equation (4). The error process $\{\epsilon_t\}$ in Equation (3) is assumed to follow an $\text{ARMA}(p, q)$ process with unknown orders p and q . Because Lund et al. (2023) fitted an $\text{AR}(1)$ model to this series, we restrict the candidate AR orders to $p \in \{0, 1, 2, 3\}$ and set $q = 0$ to reduce computational complexity. For the constant mean shift and fixed common-trend mean shift models, the BIC objective function `ar_bic_mean` is used with different design matrices to reflect the corresponding mean structures. For the trend shift model, the BIC objective function `ar_bic_mean_trend_shift` is used. All three objective functions are provided in Section S2 of the supplementary materials. The `cptgaisl` method with `minDist=2` is then applied to each model to jointly select the AR order and detect changepoints. The results are reported in Table 6. Additionally, the output is displayed below only for the fixed common-trend mean shift model.

```

prange <- list(ar = c(0, 3))

XMat_mean_fixedtrend <- cbind(1, (1:N)/N)
res_mean_fixedtrend <- cptgaisl(
  ObjFunc = ar_bic_mean, N = N, prange = prange, popSize = 400, numIslands = 10,
  minDist = 2,
  pcrossover = 0.95, pmutation = 0.3, pchangept = 10 / N, option = "both",
  XMat = XMat_mean_fixedtrend, Xt = Xt, seed = 123456
)
summary(res_mean_fixedtrend)

#> #####
#> #   Changepoint Detection via Island Model GA   #
#> #####
#>   Settings:
#>   Population size           = 400
#>   Number of Island         = 10
#>   Island size               = 40
#>   Number of generations    = 5050
#>   Number of migrations     = 101
#>   Crossover probability     = 0.95
#>   Mutation probability     = 0.3
#>   Changepoint probability  = 0.07462687
#>   minDist                  = 2
#>   Task mode                 = both
#>   Parallel Usage           = FALSE
#>   Seed                     = 123456
#>
#> ##### Island Model GA results #####
#>   Optimal Fitness value = 247.2428
#>   Optimal Solution:
#>     Number of Changepoints = 2
#>     Model hyperparameters:
#>       ar = 0
#>     Changepoints Locations = 79 105

```

Table 6: BIC-based changepoint model selection results via the island model genetic algorithm for the Atlanta airport temperature series.

Models	BIC	Detected changepoints	Detected AR order
Constant mean shift model	252.9865	1918, 1960, 1983	0
Fixed common-trend mean shift model	247.2428	1957, 1983	0
Trend shift model	248.3588	1959	0

The estimated value of p is zero for all three models, indicating that the fitted error process ϵ_t is uncorrelated. Among the three candidate models, the fixed common-trend mean shift model has the smallest BIC value, 247.2428. This model detects $\hat{m} = 2$ changepoints, located at $\hat{\tau} = \{79, 105\}$, corresponding to the years 1957 and 1983, as shown in the R output above. This illustrates that multiple changepoint detection procedures often flag one or more changepoints in an attempt to track the series mean when a common trend is present but ignored (Lund et al., 2023). By contrast, under a piecewise constant mean shift model with an AR(1) error process, Lund et al. (2023) identified three changepoints in 1921, 1960, and 1984 using the GA package, with BIC = 253.5776. The changepointGA analysis recovers two changepoints close to the later two changepoints reported by Lund et al. (2023). The smaller BIC under the fixed common-trend specification with uncorrelated errors suggests that allowing a common trend and jointly selecting the AR order and changepoint locations provides a better fit for this series.

The sample autocorrelation function (ACF) and partial autocorrelation function (PACF) plots of the residuals from the fixed common-trend mean shift model, with two estimated changepoints at $\hat{\tau} = \{79, 105\}$, are shown in Figure 9. These diagnostics suggest that the fitted model adequately captures the dependence structure in the series. In addition, Figure 10 displays the detected changepoints, shown as blue dashed vertical lines, and the estimated level shifts around the common trend, shown as red dashed lines, overlaid on the time series. The estimated common trend parameter is $\hat{\alpha}_0 = 1.5059$, with a standard error of 0.3400.

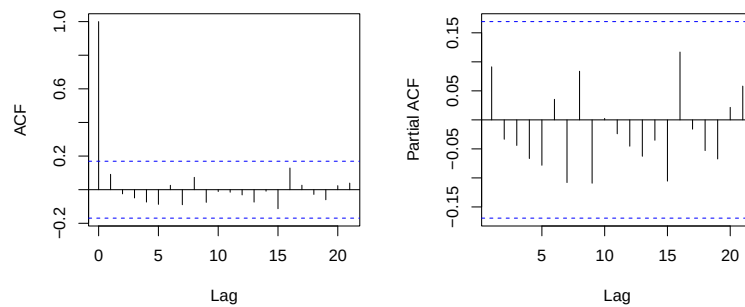


Figure 9: Sample ACF and PACF plots of the residuals from the fixed common-trend mean shift model with two estimated changepoints by changepointGA.

```
arcptfit <- ar_bic_mean(chromosome=res_mean_fixedtrend@overbestchrom,
                      XMat=XMat_mean_fixedtrend, Xt=Xt)
DesignX <- attributes(arcptfit)$DesignX
fit <- attributes(arcptfit)$fit
beta_xreg <- tail(fit$coef, ncol(DesignX))
xbeta <- as.numeric(DesignX %*% beta_xreg)
plot(res_mean_fixedtrend, data = Xt, show_segmean = FALSE,
     XTickLab = 1879:2012, XTickPos = seq(1880, 2010, by = 10),
     XAxisLab = "Year", YAxisLab = "Yearly Average Temperature")
lines(1879:2012, xbeta, col = "red", lwd = 2.5, lty = "dashed")
```

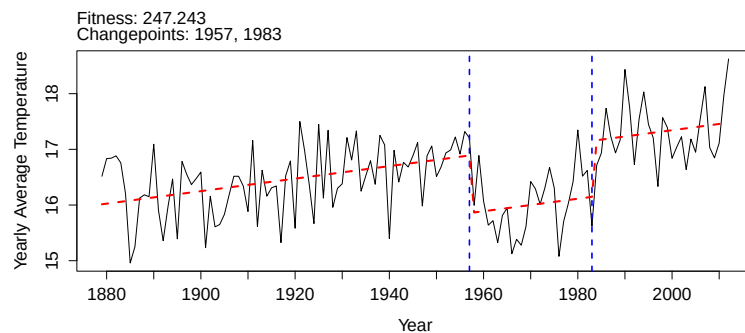


Figure 10: A changepoint analysis of the Atlanta airport temperature series.

6 Conclusion

This paper presents the R package `changepointGA`, which applies genetic algorithm optimization to changepoint detection problems. The package provides substantial flexibility, allowing users to customize objective functions and genetic operators, enabling experimentation and extension.

Compared with the `GA` package, `changepointGA` differs in three main respects. First, it employs a more compact chromosome representation for long time series, using a parameter vector whose dimensionality adapts to the chromosome configuration, whereas `GA` relies on binary encoding. The choice of chromosome representation plays an important role in the performance of genetic algorithms. Second, `changepointGA` enforces a minimum segment length, preventing unrealistic changepoint configurations permitted in the `GA` package. Third, it enables joint estimation of time series AR and MA orders and changepoint locations, ensuring identification under the best-fitting model specification. Although `changepointGA` may sacrifice some precision in achieving global optimality, it provides a computationally efficient stochastic search for changepoint detection.

We believe that the `changepointGA` package will benefit the research community by providing a user-friendly, accurate, and adaptable tool for a wide range of changepoint detection applications. Its versatility makes it applicable to diverse domains, including climate studies, genetic research, quality control, and other areas requiring precise and efficient changepoint detection.

References

- E. Andreou and E. Ghysels. Detecting multiple breaks in financial market volatility dynamics. *Journal of Applied Econometrics*, 17(5):579–600, 2002. doi: 10.1002/jae.684. [p1]
- A. Aue and L. Horváth. Structural breaks in time series. *Journal of Time Series Analysis*, 34(1):1–16, 2013. doi: 10.1111/j.1467-9892.2012.00819.x. [p1]
- J. Bai and P. Perron. Estimating and testing linear models with multiple structural changes. *Econometrica*, 66(1):47–78, 1998. doi: 10.2307/2998540. [p1]
- J.-L. Chapman, I. Eckley, and R. Killick. A nonparametric approach to detecting changes in variance in locally stationary time series. *Environmetrics*, 31(1):e2576, 2020. doi: 10.1002/env.2576. [p1]
- J. Chen and A. K. Gupta. *Parametric Statistical Change Point Analysis*. Birkhäuser Boston, Boston, MA, 2000. doi: 10.1007/978-1-4757-3131-6. [p1]
- L. Davis. *Handbook of Genetic Algorithms*. Van Nostrand Reinhold, New York, 1991. [p3]
- R. A. Davis, T. C. M. Lee, and G. A. Rodriguez-Yam. Structural Break Estimation for Nonstationary Time Series Models. *Journal of the American Statistical Association*, 101(473):223–239, 2006. doi: 10.1198/016214505000000745. [p1, 2]
- H. Dette, W. Wu, and Z. Zhou. CHANGE POINT ANALYSIS OF CORRELATION IN NON-STATIONARY TIME SERIES. *Statistica Sinica*, 29(2):611–643, 2019. URL <https://www.jstor.org/stable/26705481>. [p1]
- P. H. C. Eilers and R. X. de Menezes. Quantile smoothing of array CGH data. *Bioinformatics*, 21(7):1146–1153, 2005. doi: 10.1093/bioinformatics/bti148. [p17]
- S. Gavrin, D. Murzagulov, and A. Zamyatin. Detection of Change Point in Process Signals by Cascade Classification. In *2018 International Russian Automation Conference (RusAutoCon)*, pages 1–4. IEEE, 2018. doi: 10.1109/RUSAUTOCON.2018.8501677. [p1]
- E. Gombay, L. Horváth, and M. Husková. ESTIMATORS AND TESTS FOR CHANGE IN VARIANCES. *Statistics & Decisions*, 14(2):145–160, 1996. doi: 10.1524/strm.1996.14.2.145. [p1]
- J. H. Holland. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. University of Michigan Press, Ann Arbor, 1975. [p1]
- L. Hsu, S. G. Self, D. Grove, T. Randolph, K. Wang, J. J. Delrow, L. Loo, and P. Porter. Denoising Array-Based Comparative Genomic Hybridization Data Using Wavelets. *Biostatistics*, 6(2):211–226, 2005. doi: 10.1093/biostatistics/kxi004. [p17]
- P. Hupé, N. Stransky, J.-P. Thiery, F. Radvanyi, and E. Barillot. Analysis of array CGH data: From signal ratio to gain and loss of DNA regions. *Bioinformatics*, 20(18):3413–3422, 2004. doi: 10.1093/bioinformatics/bth418. [p17]
- R. Killick and I. A. Eckley. changepoint: An R package for changepoint analysis. *Journal of Statistical Software*, 58(3):1–19, 2014. URL <https://www.jstatsoft.org/index.php/jss/article/view/v058i03>. [p16, 17]
- W. R. Lai, M. D. Johnson, R. Kucherlapati, and P. J. Park. Comparative Analysis of Algorithms for Identifying Amplifications and Deletions in Array CGH Data. *Bioinformatics*, 21(19):3763–3770, 2005. doi: 10.1093/bioinformatics/bti611. [p16, 17]
- J. Lee, S. Li, and R. Lund. Trends in Extreme U.S. Temperatures. *Journal of Climate*, 27(11):4209–4225, 2014. doi: 10.1175/JCLI-D-13-00283.1. [p2]
- J. Lee, R. Lund, J. Woody, and Y. Xu. Trend assessment for daily snow depths with changepoint considerations. *Environmetrics*, 31(1):e2580, 2020. doi: 10.1002/env.2580. [p1]
- M. Li and Q. Lu. Changepoint detection in autocorrelated ordinal categorical time series. *Environmetrics*, 33(7):e2752, 2022. doi: 10.1002/env.2752. [p1]
- M. Li, Q. Lu, and X. Wang. Changepoint Detection in Categorical Time Series with Application to Daily Total Cloud Cover in Canada, 2026. URL <https://arxiv.org/abs/2605.20621>. [p2]
- Q. Lu, R. Lund, and T. C. M. Lee. An MDL approach to the climate segmentation problem. *The Annals of Applied Statistics*, 4(1):299–319, 2010. doi: 10.1214/09-AOAS289. [p2]

- R. B. Lund, C. Beaulieu, R. Killick, Q. Lu, and X. Shi. Good Practices and Common Pitfalls in Climate Time Series Changepoint Techniques: A Review. *Journal of Climate*, 36(23):8041–8057, 2023. doi: 10.1175/JCLI-D-22-0954.1. [p17, 18]
- E. S. Page. A Test for a Change in a Parameter Occurring at an Unknown Point. *Biometrika*, 42(3/4): 523–527, 1955. doi: 10.2307/2333401. [p1]
- P. Perron. The Great Crash, the Oil Price Shock, and the Unit Root Hypothesis. *Econometrica*, 57(6): 1361–1401, 1989. doi: 10.2307/1913712. [p2]
- T. Polushina and G. Sofronov. Change-point detection in biological sequences via genetic algorithm. In *2011 IEEE Congress of Evolutionary Computation*, pages 1966–1971. IEEE, 2011. doi: 10.1109/CEC. 2011.5949856. [p2]
- R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2024. URL <http://www.R-project.org/>. [p1]
- M. Robbins, C. Gallagher, R. Lund, and A. Aue. Mean shift testing in correlated data. *Journal of Time Series Analysis*, 32(5):498–511, 2011. doi: 10.1111/j.1467-9892.2010.00707.x. [p1]
- L. Scrucca. GA: A Package for Genetic Algorithms in R. *Journal of Statistical Software*, 53(4):1–37, 2013. doi: 10.18637/jss.v053.i04. [p1, 6]
- L. Scrucca. On Some Extensions to GA package: Hybrid Optimisation, Parallelisation and Islands Evolution. *The R Journal*, 9(1):187–206, 2017. doi: 10.32614/RJ-2017-008. [p1, 2]
- X. Shi, C. Beaulieu, R. Killick, and R. Lund. Changepoint Detection: An Analysis of the Central England Temperature Series. *Journal of Climate*, 35(19):6329–6342, 2022. doi: 10.1175/JCLI-D-21-0489.1. [p1, 2]
- R. Shibata. Selection of the order of an autoregressive model by Akaike’s information criterion. *Biometrika*, 63(1):117–126, 1976. doi: 10.1093/biomet/63.1.117. [p2]
- P. Wang, Y. Kim, J. Pollack, B. Narasimhan, and R. Tibshirani. A method for calling gains and losses in array CGH data. *Biostatistics*, 6(1):45–58, 2005. doi: 10.1093/biostatistics/kxh017. [p17]
- Y.-C. Yao. Estimating the number of change-points via Schwarz’ criterion. *Statistics & Probability Letters*, 6(3):181–189, 1988. doi: 10.1016/0167-7152(88)90118-6. [p2]
- A. Zeileis, F. Leisch, C. Kleiber, and K. Hornik. Monitoring structural change in dynamic econometric models. *Journal of Applied Econometrics*, 20(1):99–121, 2005. doi: 10.1002/jae.776. [p1]
- N. R. Zhang and D. O. Siegmund. A Modified Bayes Information Criterion with Applications to the Analysis of Comparative Genomic Hybridization Data. *Biometrics*, 63(1):22–32, 2007. doi: 10.1111/j.1541-0420.2006.00662.x. [p2]
- E. Zivot and D. W. K. Andrews. Further Evidence on the Great Crash, the Oil-Price Shock, and the Unit-Root Hypothesis. *Journal of Business & Economic Statistics*, 10(3):251–270, 1992. doi: 10.1080/07350015.1992.10509904. [p2]

Mo Li
 University of Louisiana at Lafayette
 Department of Mathematics
 217 Maxim Doucet Hall, Lafayette, LA 70504,
 United States of America
mo.li@louisiana.edu

QiQi Lu
 Virginia Commonwealth University
 Department of Statistical Sciences and Operations Research
 PO Box 843083, 1015 Floyd Ave., Richmond, VA 23284,
 United States of America
qlu2@vcu.edu